Python Engineering - Class 03 Reference Guide

Loops and Iteration

Reference guide to the syntax and tools used in this class. It explains what each element does, how it is written and when it is useful.

Quick map

Concept	Syntax	Idea
for	for i in range(...):	Use when the number or sequence of iterations is controlled by an iterable.
while	while condition:	Use when repetition continues until a condition changes.
range()	range(start, stop, step)	Generates integer sequence; stop is excluded.
Accumulator	total += value	Stores a running sum/product or evolving result.
Counter	i += 1	Tracks number of iterations when needed.
break / stopping rule	while h >= 0	Numerical simulations need a clear termination criterion.
Integer division	//	Useful when an exact integer result is intended.
math.factorial()	factorial(n)	Reference implementation used for validation.
math.exp()	exp(x)	Reference exponential used to validate Taylor approximation.

1. for

Syntax: `for i in range(...):`

Use when the number or sequence of iterations is controlled by an iterable.

```
for i in range(1, N + 1):
    total += i**2
```

Control question: before running the example, predict the result type and shape.

2. while

Syntax: `while condition:`

Use when repetition continues until a condition changes.

```
while h >= 0:
    ...
```

Control question: before running the example, predict the result type and shape.

3. range()

Syntax: `range(start, stop, step)`

Generates integer sequence; stop is excluded.

```
for i in range(1, 6):
    print(i)
```

Control question: before running the example, predict the result type and shape.

4. Accumulator

Syntax: `total += value`

Stores a running sum/product or evolving result.

```
total = 0
for x in values:
    total += x
```

Control question: before running the example, predict the result type and shape.

5. Counter

Syntax: `i += 1`

Tracks number of iterations when needed.

```
i = 0
i += 1
```

Control question: before running the example, predict the result type and shape.

6. break / stopping rule

Syntax: `while h >= 0`

Numerical simulations need a clear termination criterion.

```
while h >= 0:
    ...
```

Control question: before running the example, predict the result type and shape.

7. Integer division

Syntax: `//`

Useful when an exact integer result is intended.

```
exact_integer = numerator // denominator
```

Control question: before running the example, predict the result type and shape.

8. math.factorial()

Syntax: `factorial(n)`

Reference implementation used for validation.

```
from math import factorial
reference = factorial(5)
```

Control question: before running the example, predict the result type and shape.

9. math.exp()

Syntax: `exp(x)`

Reference exponential used to validate Taylor approximation.

```
from math import exp
reference = exp(0.5)
```

Control question: before running the example, predict the result type and shape.

Common mistakes

- Confusing a function name with a function call using parentheses.
- Not checking types, indices or shapes before an operation.
- Using a tool because it works without being able to explain what it receives and returns.

Mental cheat sheet

```
for: for i in range(...):
while: while condition:
range(): range(start, stop, step)
Accumulator: total += value
Counter: i += 1
break / stopping rule: while h >= 0
Integer division: //
math.factorial(): factorial(n)
math.exp(): exp(x)
```